
Proyecto Final Machine Learning

Mateo Gómez-Acebo Girardet
Machine Learning 2025/2026
Universidad Pontificia Comillas

Abstract

Análisis de `rendimiento_estudiantes.csv` (4.424 estudiantes) abordando clasificación multiclase, regresión y aprendizaje no supervisado para comprender las trayectorias académicas en educación superior. El rendimiento del primer semestre emerge como señal dominante en todas las tareas; el abandono resulta estructuralmente impredecible sin variables contemporáneas del segundo semestre.

1 Análisis descriptivo del conjunto de datos

Vamos a trabajar con el dataset `rendimiento_estudiantes.csv` que contiene información sobre **4.424 estudiantes de educación superior**. Tenemos 37 columnas, de las cuales **36 son variables de entrada (X) y la última es la variable objetivo (y)**. Los inputs cubren varios bloques importantes de información sobre los estudiantes:

Perfil demográfico y socioeconómico	genero, edad_al_matricularse, nacionalidad, desplazado, internacional, estado_civil, becado, deudor, matricula_al_dia, necesidades_educativas_especiales, tasa_desempleo, tasa_inflacion, pib, cualificacion_madre, cualificacion_padre, ocupacion_madre, ocupacion_padre
Contexto académico previo al ingreso	modo_solicitud, orden_solicitud, curso, cualificacion_previa, nota_cualificacion_previa, nota_admision
Rendimiento académico universitario	asignaturas_1sem_matriculadas, _evaluadas, _aprobadas, _convalidadas, nota_media_1sem, asistencia_diurna_vespertina, _sin_evaluacion, asignaturas_2sem_matriculadas, _evaluadas, _aprobadas, _convalidadas, nota_media_2sem, _sin_evaluacion

La variable objetivo es de tipo **categorico multiclase** y distingue entre tres situaciones finales posibles del estudiante: **Graduado, Abandono y Matriculado**. Podemos ver las proporciones de alumnos en cada una de estas situaciones en el dataset:

Clase	Número de estudiantes	Porcentaje
Graduado	2 209	49,9 %
Abandono	1 421	32,1 %
Matriculado	794	17,9 %

Las clases no están equilibradas — **este desbalance relevante será tratado en la siguiente fase.**

2 Preprocesado

Verificación de valores nulos. Constatamos que **no existe ningún valor nulo en ninguna de las 37 columnas**. Esto no es típico cuando tenemos datasets tan grandes y nos permite evitar usar estrategias de imputación.

Tratamiento de outliers. La **variable con casos más llamativos de outliers es edad_al_matricularse** ya que su rango va de 17 a 70 años con mucha asimetría positiva. Una edad de 70 años al matricularse puede alterar nuestro modelo. En nuestro notebook encontramos que la asimetría no se consigue corregir correctamente en esa variable debido a esos valores. **Decidimos entonces eliminar los outliers del conjunto de entrenamiento** para no comprometer los modelos sensibles a la escala.

Codificación de variables categóricas. Tenemos 17 variables de tipo string. Usaremos **Label Encoding (0/1)** para las que solo toman dos valores: desplazado, necesidades_educativas_especiales, deudor, matricula_al_dia, genero, becado, internacional, asistencia_diurna_vespertina. Para las variables con muchas categorías usaremos **One-Hot-Encoding** con `drop_first=True`: estado_civil, modo_solicitud, curso, cualificacion_previa. Sin embargo, hay varias columnas con muchísimas categorías (46 en ocupacion_padre) y con One-Hot generaríamos muchísimas columnas que podría perjudicar nuestro modelo. Por eso **agrupamos categorías semánticamente** para: nacionalidad, cualificacion_madre, cualificacion_padre, ocupacion_padre, ocupacion_madre.

Asimetría en variables continuas. Muchos algoritmos de ML asumen cierta simetría en la distribución de las variables de entrada. Si la asimetría es elevada puede distorsionar modelos lineales basados en distancias como KNN. Por ello usamos la **transformación Box-Cox** para corregir este problema. Sin embargo, Box-Cox solo funciona con valores positivos pero `nota_media_1sem` tiene valores 0 así que en ese caso usamos la variante **Yeo-Johnson**. Las columnas con asimetrías superiores a (± 1) son: `edad_al_matricularse`, `nota_media_1sem`, `nota_media_2sem`.

Escalado (Estandarización). Las variables numéricas presentan escalas muy distintas y eso puede afectar a algoritmos de distancias como KNN, por ello usaremos **StandardScaler** (media 0, desviación típica 1) sobre todas las variables numéricas continuas. **El escalado se hace sobre los datos de entrenamiento y luego se aplica sobre train, validación y test por separado** para evitar Data Leakage.

Exclusión de variables por Data Leakage. En regresión queremos predecir `nota_media_2sem` pero para que el modelo no use variables contemporáneas que equivaldrían a mirar la respuesta antes del examen necesitamos excluir: `asignaturas_2sem_evaluadas` (para conocer la nota hay que haber sido evaluado), `asignaturas_2sem_aprobadas` (variable consecuencia directa de la nota media), `asignaturas_2sem_sin_evaluacion` (información generada al mismo tiempo que la nota), `asignaturas_2sem_convalidadas` (información del 2do semestre, no disponible a priori), `asignaturas_2sem_matriculadas` (en la práctica va ligado al resultado final).

Tratamiento de desbalanceo de clases. La clase “Matriculado” está infrarrepresentada (18%). La estrategia que usaremos será el uso de `class_weight=“balanced”` disponible en los modelos de sklearn. Con ese parámetro no modificamos los datos sino que ajustamos internamente los pesos de cada clase de forma inversamente proporcional a su frecuencia. Como exploración adicional, **compararemos este enfoque con SMOTE** que genera ejemplos sintéticos de la clase minoritaria (sobre el conjunto de entrenamiento solo). No consideramos el undersampling ya que el tamaño del dataset es relativamente moderado y la eliminación de muestras reales puede suponer una pérdida de información difícil de justificar.

División Train / Validación / Test. Dividimos el dataset en tres conjuntos: entrenamiento (70%), validación (15%) y test (15%). Usaremos `stratify=objetivo` garantizando que la proporción de las clases se mantenga en todos los conjuntos, con `shuffle` aleatorio previo.

El archivo `preprocessing.py` genera: `X_train.csv` / `X_test.csv` (clasificación y no supervisado), `X_train_reg.csv` / `X_test_reg.csv` (regresión) e `y_train.csv` / `y_test.csv`. Hemos creado además `resumen_preprocesado.ipynb` con las especificaciones de las agrupaciones y gráficos antes/después del preprocesado.

3 Clasificación

3.1 Regresión Logística

Es el primer modelo que vamos a entrenar porque actúa como línea base, devuelve la probabilidad de pertenencia a cada clase. Además, es muy interpretable ya que los coeficientes aprendidos nos dicen directamente qué variables empujan al estudiante hacia cada clase.

Decisiones de diseño. Con 98 features el modelo tiene mucho riesgo de sobreajuste, por lo que usamos regularización (Ridge y Lasso) + versión multinomial + `class_weight='balanced'`. La búsqueda de hiperparámetros se realiza mediante GridSearchCV $k=5$, probando 6 valores de C en escala logarítmica $\times 2$ tipos de regularización = 60 entrenamientos. Desarrolladas en detalle en `decisiones_diseno.ipynb`.

Hiperparámetros óptimos: Lasso con $C = 0,1$. La elección de una regularización tan fuerte no es un resultado trivial: apunta directamente a la existencia de **redundancia entre features**, hipótesis que confirmamos posteriormente al analizar la colinealidad y detectar correlaciones altas entre variables académicas del mismo semestre. El análisis de hiperparámetros no solo optimiza el rendimiento, actúa también como diagnóstico del dataset revelando estructura que había pasado desapercibida en el preprocesado.

Rendimiento por clase: resultados sólidos en ‘Abandono’ ($F_1 = 0,764$) y ‘Graduado’ ($F_1 = 0,840$). ‘Matriculado’ obtiene el F_1 más bajo (0,537), pero esto **no es un fallo del modelo sino un reflejo de la naturaleza ambigua de la clase**: los alumnos matriculados son observaciones en tránsito cuyos perfiles académicos solapan simultáneamente con los de abandono y los de graduado. Ningún clasificador puede separar bien clases que no son separables en el espacio de features.

Interpretabilidad de coeficientes: el análisis de los coeficientes de la clase ‘Matriculado’ (la más problemática) permite extraer conclusiones con valor académico y sociológico real. **La evaluación en el 2.º semestre emerge como el indicador más claro de continuidad del alumno**, y el nivel educativo de los padres muestra una influencia medible sobre la probabilidad de graduarse. La regresión logística no es solo el modelo más interpretable del proyecto, es la herramienta que más enriquece la comprensión del dataset.

3.2 KNN

A diferencia de regresión logística, KNN es **no paramétrico y no tiene una fase de entrenamiento real** ya que simplemente memoriza todos los datos de train y cuando recibe un nuevo alumno busca los K vecinos más cercanos y le asigna la clase mayoritaria entre ellos. La distancia que usamos para medir la cercanía entre puntos es la distancia Minkowski que representa la distancia Manhattan si usamos $p=1$ y la euclídea si $p=2$. Tenemos entonces **dos hiperparámetros importantes**: $K =$ **cuántos vecinos consultar**, $p =$ **qué noción de distancia usar**.

Decisiones de diseño. KNN necesita features con misma escala debido a que usa distancias — ya resuelto en el preprocesado con StandardScaler. Usamos `weights='distance'` para mitigar el desbalanceo. GridSearchCV $k=5$, $K \in \{1, 3, 5, 7, 11, 15, 21, 31\} \times p \in \{1, 2\} = 80$ entrenamientos. Desarrolladas en detalle en `decisiones_diseno.ipynb`.

Hiperparámetros óptimos: $K = 7$, distancia Manhattan ($p = 1$). Macro F_1 en test: **0,584** — muy por debajo de Regresión Logística (0,714). Los resultados revelan cuatro factores estructurales que explican este rendimiento y que son en sí mismos hallazgos interpretativos sobre el dataset:

- **Limitación del mecanismo de desbalance:** `weights='distance'` pondera el voto de cada vecino pero no redistribuye la distribución de clases en el vecindario. Su efecto corrector es mucho menor que `class_weight='balanced'` en regresión logística, que actúa sobre la función de pérdida global.
- **Solapamiento geométrico intrínseco:** ‘Matriculado’ y ‘Abandono’ ocupan regiones del espacio de features que se solapan con ‘Graduado’. Este solapamiento, ya detectado en el modelo anterior, aquí se agrava: un alumno matriculado con actividad académica alta es geométricamente indistinguible de un graduado, y uno con perfil débil se confunde con abandono.

- **Maldición de la dimensionalidad:** con 98 features las distancias entre puntos tienden a igualarse y el concepto de ‘vecino cercano’ pierde significado. Esto se confirma visualmente al analizar la distribución de distancias al séptimo vecino en el notebook.
- **Multicolinealidad no resuelta:** la alta correlación entre variables académicas del mismo semestre, que Lasso corrige automáticamente anulando coeficientes redundantes, infla artificialmente el peso de la dimensión académica en el cálculo de distancias. Las features socioeconómicas quedan tapadas, y perfiles de ‘Matriculado’ con actividad académica alta parecen más cercanos a ‘Graduado’ de lo que realmente son.

Adicionalmente, las clases minoritarias muestran una gran dispersión geométrica, lo que aumenta aún más la dificultad de clasificarlas correctamente. **KNN no es adecuado para este problema.**

3.3 Random Forest

Random Forest aparece como tercer y último modelo de este apartado de clasificación y su inclusión en el proyecto en este punto tiene explicación. Da respuestas directas a varias limitaciones que hemos identificado en los dos modelos anteriores: regresión logística funcionó muy bien pero asumía relaciones lineales entre features y clases; KNN sufrió la maldición de la dimensionalidad y la multicolinealidad de las variables académicas. Random Forest aborda ambos problemas. Es un **método de ensemble** basado en árboles de decisión que particionan el espacio en regiones rectangulares con preguntas binarias. Un árbol tiene problemas de sobreajuste al train y es especialmente sensible a pequeñas variaciones en los datos. Por tanto Random Forest construye muchos árboles independientes y el **truco fundamental reside en la decorrelación de árboles = el split no se busca entre las 98 features sino entre un subconjunto aleatorio**. Eso hace que los árboles no compartan siempre las mismas features dominantes y por tanto se genere una diversidad real.

Decisiones de diseño. Hiperparámetros clave: `n_estimators`, `max_features` y `max_depth`. Para el desbalance, `class_weight='balanced'` actúa igual que en regresión logística pero árbol a árbol. GridSearchCV $k=5$ sobre $\{100, 300\} \times \{\text{"sqrt"}, 20\} \times \{\text{None}, 20\} = 40$ entrenamientos (grid limitado por coste computacional). Desarrolladas en detalle en `decisiones_diseno.ipynb`.

Hiperparámetros óptimos: `n_estimators=300`, `max_features=20`, `max_depth=20`. Macro F_1 test: 0,700 · Accuracy: 0,770. La elección de `max_features=20` en lugar del estándar $\sqrt{98} \approx 10$ tiene una explicación directa en la estructura del dataset: con solo 10 features por split, el subconjunto aleatorio tiene alta probabilidad de estar dominado por features correlacionadas del mismo bloque semántico (variables académicas del mismo semestre). Con 20 features, cada split tiene más probabilidad de ver features de bloques distintos, tomando decisiones menos sesgadas por la colinealidad. El trade-off —árboles algo más correlacionados entre sí pero individualmente más informativos— resulta favorable en este caso.

Random Forest resuelve los dos problemas estructurales identificados anteriormente: elimina el sesgo geométrico de KNN (no trabaja con distancias locales) y supera la linealidad de Regresión Logística capturando relaciones no lineales que elevan el F_1 de ‘Graduado’ hasta 0,860. La selección aleatoria de features regula la colinealidad de forma implícita, y la **profundidad limitada a 20 actúa como regularización suave** que evita el sobreajuste sin sacrificar capacidad expresiva.

Feature importance: `asignaturas_2sem_aprobadas` lidera el ranking como la variable más discriminativa, confirmando lo que ya se intuía en los coeficientes de regresión logística para la clase ‘Matriculado’.

Limitación importante: `class_weight='balanced'` pierde efectividad en un ensemble. En Regresión Logística modifica la función de pérdida de forma global durante todo el ajuste; en Random Forest actúa árbol a árbol y el promediado diluye su efecto. Esto ilustra que **mayor complejidad de modelo no garantiza mejor Macro F_1** cuando la métrica penaliza por igual las tres clases y el desbalance no se gestiona con igual eficacia.

	Reg. Logística	KNN	Random Forest
Macro F1 (CV)	0.706	0.561	0.687
Macro F1 (test)	0.714 ★	0.584	0.700
Accuracy (test)	0.713	0.685	0.770 ★
F1 abandono	0.764	0.623	0.765
F1 graduado	0.840	0.795	0.860 ★
F1 matriculado	0.537 ★	0.334	0.473

★ indica el mejor valor por métrica. El Macro F1 penaliza el desbalanceo entre clases; la Accuracy favorece a las clases mayoritarias.

Tabla comparativa final — Modelos de Clasificación

4 Regresión

En regresión cambiamos la tarea objetivo de clasificación de manera significativa ya que nos vamos a centrar en predecir `nota_media_2sem` que representa la calificación media por estudiante en el segundo semestre. La gran diferencia con clasificación es que **ya no predecimos una categoría sino un valor numérico continuo con un rango acotado**. El modelo que vamos a usar es la regresión lineal que asume una relación entre features y el target aproximadamente lineal y termina obteniendo un coeficiente por cada variable de entrada.

El target ahora es distinto al que teníamos en el apartado anterior y eso nos expone a tener que volver a considerar de nuevo si la información que nos servía en clasificación es directamente aplicable a regresión lineal. La respuesta es clara: no es posible ya que el target es `nota_media_2sem` y está muy relacionado con otras variables del segundo semestre. Por tanto, **usar esas variables como predictores equivaldría a incluir en el modelo información que en realidad se conoce después del target** y eso puede producir métricas artificialmente buenas en train pero que terminan demostrando la inutilidad del modelo en test. Esto se conoce como data leakage y en la parte de preprocesado ya resolvimos este problema preparando el terreno para cada tipo de problema pedido en el enunciado. Por tanto, tenemos dos archivos `X_train_reg.csv` y `X_test_reg.csv` que han sido generados expresamente sin esas 5 columnas del 2do semestre problemáticas.

Por tanto, la pregunta que respondemos en esta segunda parte del proyecto es: **¿hasta qué punto es posible anticipar el rendimiento del 2do semestre a partir del perfil de entrada del alumno, su contexto socioeconómico y su desempeño en el 1er semestre?** A diferencia de clasificación, donde el desbalanceo entre clases era uno de los principales retos a la hora de diseñar el modelo, en regresión ya no existe ese problema ya que tenemos un target numérico continuo.

Decisiones de diseño. Entrenamos dos modelos: **Ridge** (distribuye pesos entre features correlacionadas) y **Lasso** (selección automática de variables). Métrica principal: **RMSE** (penaliza errores grandes) + R^2 . GridSearchCV $k=5$, $\alpha \in \{0,001, 0,01, 0,1, 1, 10, 100\} \times 2$ modelos = 60 entrenamientos. Desarrolladas en detalle en `decisiones_diseno.ipynb`.

Nota. Durante la implementación del modelo de regresión detectamos un caso real de data leakage: introdujimos sin querer `nota_media_2sem` como predictor siendo esta la propia variable objetivo. El modelo obtuvo métricas perfectas que nos alertaron del problema. Fue corregido.

Modelo óptimo: Lasso con $\alpha = 0,1 \rightarrow R^2 = 0,749$. El $\alpha = 100$ óptimo para Ridge confirma la colinealidad elevada ya identificada en clasificación: Ridge necesita penalizar muy fuerte para repartir el peso entre features correlacionadas. Que el modelo óptimo sea Lasso cuadra perfectamente con la naturaleza del dataset — su selección automática de variables elimina los redundantes y concentra la señal en los predictores realmente informativos. La caída brusca del rendimiento de Lasso al regularizar fuerte delata además que **la señal predictiva está distribuida entre muchas variables con contribuciones moderadas**: en cuanto se eliminan, el error aumenta de forma significativa.

Hallazgo diagnóstico clave — la columna en $x = 0$: el análisis de predicciones vs valores reales revela una columna vertical de errores grotescos en $x = 0$. Estos corresponden a alumnos que abandonaron sin presentarse al 2.º semestre. Su perfil del 1.er semestre es indistinguible del de un alumno normal, lo que demuestra que **el abandono no es predecible con las variables**

disponibles antes del 2.º semestre. Este problema no existía en clasificación porque allí sí teníamos las variables del 2.º semestre, que aumentan enormemente la capacidad discriminativa. Las variables socioeconómicas, aunque valiosas, no capturan decisiones personales o circunstancias externas que determinan el abandono real.

La asimetría negativa de los residuos es la firma estadística de este mismo problema. Eliminar los alumnos con `nota_media_2sem= 0` debería corregirla y mejorar las métricas.

Interpretación de coeficientes — dos niveles de señal predictiva:

- **Nivel 1 (dominante):** `nota_media_1sem` y `asignaturas_1sem_aprobadas`. El rendimiento del 1.er semestre es con diferencia el mejor predictor del 2.º. Existe una continuidad académica clara y medible.
- **Nivel 2 (colectivamente igual de importante):** un conjunto de variables socioeconómicas — `matricula_al_dia` (estrés económico), género (mayor consistencia intersemestral en mujeres, patrón típico del sistema educativo europeo) — con contribuciones individuales moderadas pero cuya importancia conjunta es clave: cuando Lasso las elimina al regularizar fuerte, el error sube bruscamente. **Que Lasso las mantenga activas confirma que el rendimiento universitario no es puramente académico:** el contexto económico y personal del alumno tiene un efecto real y medible sobre sus resultados.

5 Implementaciones adicionales

5.1 Análisis comparativo entre `class_weight='balanced'` y SMOTE

Hemos implementado un código y un jupyter notebook en el que reentrenamos los modelos de Regresión logística y Random Forest pero usando SMOTE para tratar el desbalanceo. Esto se nos ha ocurrido debido a que en varios modelos anteriores pensábamos que el efecto de la técnica `class_weight` podría estar afectando negativamente a nuestros resultados. Todas las conclusiones y análisis de las ideas adicionales figuran en el notebook `resumen_smote_adicional.ipynb`.

6 Aprendizaje no supervisado

En las dos secciones anteriores hemos estado trabajando con aprendizaje supervisado: el modelo sabía qué variable predecir y aprendía a partir de etiquetas conocidas. Con el aprendizaje no supervisado cambiamos el enfoque del problema, ya no tenemos una variable target que predecir, ahora queremos encontrar una estructura en los datos como por ejemplo patrones o agrupaciones que nos permitan sacar conclusiones interesantes sobre la geometría del espacio de features.

Por lo tanto en esta sección nos planteamos lo siguiente: ¿existen perfiles de estudiantes que se agrupan de forma natural más allá de las tres categorías que predecíamos antes? Esta pregunta y muchas otras en relación con la estructura misma del espacio de features no tienen una respuesta directa en los modelos supervisados con los que hemos trabajado hasta entonces. Por lo tanto, el aprendizaje no supervisado aporta un enorme valor en este aspecto, permitiendo entender y visualizar mejor el espacio X que suele ser un trabajo que precede la implementación de los modelos implementados en apartados anteriores.

Decisiones de diseño:

Combinamos PCA y K-Means de forma secuencial. PCA reduce la dimensionalidad eliminando ruido y colinealidad, obteniendo un espacio limpio sobre el que K-Means puede calcular distancias euclídeas significativas. Partimos de `X_train.csv` seleccionando un subconjunto razonado de 14 variables numéricas que cubren tres ejes de diferenciación: rendimiento académico universitario (asignaturas matriculadas, evaluadas, aprobadas y notas medias de ambos semestres), perfil de acceso (`nota_admision`, `nota_cualificacion_previa`, `edad_al_matricularse`) y situación económica (`becado`, `matricula_al_dia`, `deudor`). Se excluyen las ~70 columnas OHE porque su efecto acumulado en las distancias euclídeas domina sobre la señal académica real. Las variables macroeconómicas se descartan por no describir al estudiante individualmente. Las 14 variables seleccionadas ya están estandarizadas desde el preprocesado, condición necesaria para que PCA trate todas las variables en igualdad de condiciones. Para la elección de K usamos el método del codo y

el Silhouette Score (K entre 2 y 8), quedándonos con el valor que maximiza el Silhouette por ser la métrica más completa. K-Means se ejecuta con `init='k-means++'` y `n_init=10` para garantizar convergencia robusta. El análisis detallado de cada decisión figura en `decisiones_diseno.ipynb`.

Conclusión:

PCA ha reducido el espacio original de 14 variables a 6 componentes que retienen el 83,6% de la varianza total, con las dos primeras componentes acumulando el 53,2% — suficiente para que la proyección 2D sea representativa. El análisis de loadings revela que PC1 captura íntegramente el rendimiento académico universitario y PC2 una dimensión socioeconómica y de perfil de acceso completamente ortogonal. Esta ortogonalidad es especialmente valiosa porque resuelve en el espacio transformado la multicolinealidad elevada que los modelos supervisados detectaban indirectamente a través de la regularización fuerte. Que sea posible ver agrupaciones interpretables en el plano 2D antes de aplicar K-Means es ya una señal de que el espacio de features tiene estructura real.

K-Means ha encontrado espontáneamente $K=3$ como partición óptima — el mismo número que las clases supervisadas — sin ver las etiquetas en ningún momento. Esto confirma que las situaciones académicas se reflejan en la geometría del espacio de features. Los tres clusters identificados tienen perfiles claros: el Cluster 1 es el perfil de riesgo de abandono, con bajo rendimiento generalizado y condiciones socioeconómicas desfavorables (79,7% Abandono); el Cluster 2 es la trayectoria estándar, heterogénea y sin separación geométrica nítida entre Graduados y Matriculados; y el Cluster 0 es el hallazgo más interesante — un subgrupo pequeño de estudiantes de mayor edad y rendimiento muy alto que la etiquetación supervisada no distinguía y que el análisis no supervisado ha identificado de forma independiente.

El análisis de estabilidad ARI muestra una distribución bimodal (media=0,830, $\sigma=0,216$), revelando que K-Means converge a dos soluciones distintas según la inicialización. El perfil de abandono es robusto en ambas — la inestabilidad afecta únicamente a la frontera difusa entre los otros dos clusters, que es exactamente donde el espacio de features no tiene separación clara. La existencia de dos soluciones no invalida las conclusiones sino que las matiza: es en sí misma información sobre la estructura del problema.

7 Conclusiones y limitaciones

En este proyecto hemos podido demostrar que el rendimiento académico universitario es un problema difícil de tratar no por malas implementaciones de las técnicas sino por la naturaleza misma de los datos recibidos en nuestro dataset.

Como desarrolladores del proyecto, hemos tomado una decisión metodológica: no maximizar la complejidad del código ni su plena implementación manual sino exprimir al máximo la interpretabilidad de cada resultado. Con scikit-learn hoy en día es sencillo poner modelos en marcha, pero la verdadera riqueza de Machine Learning es entender por qué cada técnica funciona o falla, construyendo un relato analítico coherente durante todo el proyecto. En un primer proyecto de esta escala, comprender qué se está midiendo y por qué vale más que dominar completamente la implementación. Una vez consolidado ese entendimiento, en futuros proyectos podremos profundizar con mucha más precisión en el código.

Esta apuesta ha tenido una gran recompensa. Desde el análisis exploratorio ya nacían sospechas sobre posibles redundancias entre variables y distribuciones irregulares en las variables. El proyecto ha demostrado que esas sospechas no nacían de un ruido sin sentido sino que tienen su explicación y cada etapa del proyecto nos ha permitido confirmar o descartar las hipótesis.

El principal hallazgo que aparece repetido de formas distintas durante las etapas es una existencia de dos dimensiones estructurales en los datos: una académica muy discriminativa y una socioeconómica. La multicolinealidad era algo que no habíamos medido en el preprocesado pero que las distintas técnicas nos han podido revelar: la regularización Lasso elegida, el heatmap de correlaciones que decidimos pintar ya cuando observamos los resultados de los modelos y los ejes principales de PCA nos lo han expuesto cada uno a su manera. Además, el análisis de los pesos en variable importance, los coeficientes que han sobrevivido tras regularizar y los pesos en las componentes principales de PCA nos han confirmado que la dimensión académica y socioeconómica del estudiante son una estructura real del fenómeno.

El análisis supervisado no solo ha producido predicciones sino que nos ha ayudado a corregir lo que cojeaba en el análisis previo. Como decíamos previamente, las sospechas de redundancias se han confirmado en la elección de regularizaciones fuertes y la dificultad persistente entre modelos para la clase matriculado nos esclareció, junto con la confirmación hecha con SMOTE, que había una mala discriminación de esa clase por la naturaleza misma del problema que tratamos: Matriculado define una condición de tránsito que comparte características con graduado y abandono. Además, el fracaso de KNN nos demostró esas limitaciones de los métodos basados en distancias cuando trabajamos en altas dimensiones y nos puso sobre la mesa una hipótesis totalmente coherente con lo observado y que confirmaríamos más adelante: el espacio de features no permite una separación limpia de las clases. Sin embargo, la consistencia de las métricas sobre abandono también nos hizo intuir en que su grupo dentro del espacio de features es más fácilmente distinguible. Los resultados son coherentes, los alumnos en abandono tienen perfiles más extremos y fáciles de distinguir mientras que matriculado y graduado se solapan más al ser perfiles de alumnos en general muy similares.

Finalmente, el aprendizaje no supervisado nos ha conseguido aportar aún más confirmaciones sobre elementos que no nos paramos a comprobar en las primeras fases de análisis y que además confirman hipótesis planteadas en la parte supervisada. Con PCA conseguimos confirmar esa multicolinealidad clara de las features y esos dos grandes ejes discriminativos del dataset. Además, K-Means nos encontró el mismo número de grupos que clases que predecíamos en la sección supervisada y esa convergencia entre conclusiones entre los distintos métodos usando caminos completamente distintos nos ha dejado aún más claro que las conclusiones que hemos extraído durante el proyecto tienen un fundamento fuerte sobre los datos y la realidad. Cabe destacar que esos clusters no separan los alumnos por las clases reales y de hecho esa heterogeneidad de uno de los clusters concentrando Matriculados y Abandonos nos confirma cosas que ya veníamos deduciendo a lo largo del proyecto: una mala separabilidad geométrica de los datos por ese solapamiento pronunciado de los Matriculados. Finalmente, el análisis no supervisado nos añadió una perspectiva nueva hallando un nuevo grupo no captado anteriormente de estudiantes con gran rendimiento académico y edad considerablemente por encima de la media del dataset.

En definitiva, el análisis supervisado nos ha permitido desarrollar hipótesis sobre aspectos que se nos escapaban durante el preprocesado, mientras que el no supervisado ha venido a reforzar nuestras creencias y desvelar perspectivas estructurales que de otro modo habrían permanecido ocultas. Estas, precisamente, la naturaleza de Machine Learning como disciplina: las técnicas no compiten entre sí, sino que se complementan y solapan a lo largo de un proyecto para cooperar, desde ángulos distintos, hacia la visión más completa y realista del fenómeno que se estudia.

Resumen de hallazgos principales

Hallazgo	Evidencia supervisada	Evidencia no supervisada
Multicolinealidad severa	Lasso > Ridge, $C=0,1$ óptimo, heatmap de correlaciones	PC1 con cargas uniformes → variables redundantes
Dimensión socioeconómica real	Lasso mantiene pib, genero, matricula_al_dia	PC2 eje ortogonal e independiente de PC1
Matriculado = ambigüedad geométrica	SMOTE no ayuda, recall bajo según modelo	Cluster 2 mezcla graduados y matriculados sin frontera clara
KNN inadecuado en alta dimensión	Peor clasificador global, colapso en 'graduado'	Distancias entre clusters confirman solapamiento
Abandono y graduado tienen perfiles claros	F_1 estable y alto en ambas clases	Cluster 1 (80% abandono) y Cluster 0 (80% graduado) muy puros

Limitaciones

Las limitaciones principales son de tres tipos. El dataset cubre universidades portuguesas, lo que restringe la generalización a otros contextos educativos. Al tratarse de datos históricos, las correlaciones encontradas no implican que actuar sobre esas variables vaya a producir los mismos efectos en la práctica. Finalmente, la clase Matriculado representa una situación de tránsito sin perfil estable, lo que establece un techo real de rendimiento que ningún clasificador puede superar con las features disponibles.